数据与算法

CH02 二元关系与抽象数据类型

北京科技大学
计算机与通信工程学院
杜　冰/皇甫伟

本节内容

- C++简单回顾
- 二元关系及性质
- 数据的逻辑与存储结构
- 数据类型与抽象数据类型

C++ 简单回顾

- 虽然Java，C#，Python等高级语言也应用广泛，大量软件代码仍用C/C++编写，各类编程资源丰富
- C++支持各种简单数据类型以及抽象数据类型
- C++语言编译器发展成熟，程序执行效率高
- C++在信息科学类教育中被广泛使用
- 数据与算法本质不局限于语言，本课程采用C/C++描述

C++ 简单回顾

```
class intStack {
  public:
      intStack(int sz=10);
      ~intStack();
      void Push(int);
      int Pop();
  protected:
      int* stack;
      int top;
  private:
      int size();
};
```

- 构造函数与类同名，在对象生成时调用
- 析构函数在删除对象时调用
- 类的成员函数可以访问其成员变量
- Private 仅能在类内部访问
  protected 可被子类访问
  public大家都能访问

C++ 简单回顾

```cpp
#include <iostream>
using namespace std;

class A
{
private:
    A(): data(10) { cout << "A" << endl; }
    ~A(){ cout << "~A" << endl; }
public:
    static A& Instance()
    {
        static A a;
        return a;
    }
    void Print()
    {
        cout << data << endl;
    }
private:
    int data;
};

int main(int argc, char** argv)
{
    A &ra = A::Instance();
    ra.Print();
}
```

1 调用类A的静态成员函数 并赋值给ra

2 声明一个静态对象实例 a

3 调用类的私有构造函数，为a赋值 a = 10（静态变量），并输出 “A”
Return a

4 调用类的公共函数，输出data “10”

5 运行完毕，调用析构函数~A()，输出 “~A”

```
A
10
~A
```

C++ 简单回顾

```
intStack::intStack(int sz=10){
        stack = new int[sz];
        top = -1;
}
intStack::~intStack{
        delete[] stack;
}
void intStack::push(int k){
        stack[++top] = k; //前缀递增操作符 ++
}
int intStack::pop(){
        return stack[top--];
}
```

- new int[sz]创建长度为sz的整型数组;
- delete[]释放数组占用的内存;
- 成员函数可以访问成员变量，注意一元操作符的顺序;

```
++a;  //前置++，就是先+1，再取值操作
a++;  //后置++，就是先取值操作，再+1
--a;  //前置--，就是先-1，再取值操作
a--;  //后置--。就是先取值操作，再-1
```

C++ 简单回顾

```
…
{   intStack S1(1000);   // int sz= 1000
    S1.push(10);         //Compile Warning? Error?
    S1.push(12);
    S1.push(3.1415);
    S1.size();
}   //~intStack()被调用
…
```

```
…
{   intStack* S2 = new intStack(1000);
    S2->push(10);
    S2->push(12);
    delete S2;    //~intStack()被调用
} …
```

C++ 简单回顾

- 不愿意用指针？用引用(reference)！

```
…
{   intStack S1(1000);
    intStack& S2=S1;
    S2.push(10);
    cout<<S1.pop()<<endl; //print 10
}   //~intStack()被调用
…
```

```
int main()
{
    int a = 10;
    int& b = a;
    int& x;        //定义需要初始化
    int& y = NULL; //没有空引用
    int&& c = a;   //没有所谓的二级引用
}
```

CSDN @weixin_56935264

引用不是新定义一个变量，而是给已存在变量取了一个别名，编译器不会为引用变量开辟内存空间，它与引用的变量共用同一块内存空间

```
void func1(intStack & S){ … }
void func2(const intStack & S){ … }
```

类型 & 引用变量名(对象名) = 引用实体
注意这里的空格是可选的，即

- &符号与前后均可以有一个空格；如下：int & ra=a;
- &符号与类型挨着，如下：int& ra=a;
- &符号与引用名称挨着，如下：int &ra=a;

Const

- 常量指针：**不可以通过修改所指向的变量的值**，但是指针**可以指向别的变量**

```
int a = 5;
const int *p =&a;
*p = 20;    //error   不可以通过修改所指向的变量的值

int b =20;
p = &b; //right   指针可以指向别的变量
```

- 指针常量：指针常量的值不可修改（不能指向别的变量），但可修改它所指向的变量值

```
int a = 5;
int *const p = &a;
*p = 20;      //right 可以修改所指向变量的值

int b = 10;
p = &b;       //error 不可以指向别的变量
```

左定值，右定向，const修饰不变量 （Const 在*左还是右）

```
1  #include <iostream>
2  using namespace std;
3  namespace A
4  {
5      int a = 100;
6      namespace B            //嵌套一个命名空间B
7      {
8          int a =20;
9      }
10 }
11
12 int a = 200;//定义一个全局变量
13
14
15 int main(int argc, char *argv[])
16 {
17     cout <<"A::a ="<< A::a << endl;
18     cout <<"A::B::a ="<<A::B::a << endl;
19     cout <<"a ="<<a << endl;
20     cout <<"::a ="<<::a << endl;
21
22     int a = 30;
23     cout <<"a ="<<a << endl;
24     cout <<"::a ="<<::a << endl;
25
26     return 0;
27 }
```

将指定命名空间内的所有标识符暴露在当前的作用域内，使得在当前作用域中可以直接引用该命名空间内的任何标识符。

全局变量 a 表达为 ::a，用于当有同名的局部变量时来区别两者

```
1  A::a =100
2  A::B::a =20
3  a =200          //全局变量a
4  ::a =200
5  a =30           //局部变量a
6  ::a =200
```

本节内容

- C++简单回顾
- 二元关系及性质
- 数据的逻辑与存储结构
- 数据类型与抽象数据类型

二元关系及性质

- **集合的笛卡尔积 (Cartesian product)**
  - 集合A,B: $A=\{a_1,a_2\}$,$B=\{b_1,b_2\}$;
  - A对B的笛卡尔积，记作A×B，定义为:

    $A\times B=\{<a,b>|a\in A$ 且 $b\in B\}$
  - 则有:

    $A\times B=\{<a_1,b_1>,<a_1,b_2>,<a_2,b_1>,<a_2,b_2>\}$
  - 一般的: $A\times B\neq B\times A$, $(A\times B)\times C\neq A\times(B\times C)$;
  - 不满足交换律和结合律
  - 扩展到多个集合的笛卡尔积.

二元关系及性质

- **二元关系 (binary relation)**
  - 定义：设有集合A,B，其笛卡尔积A×B的任意一个子集 R⊆A×B，被称为A到B的一个二元关系；如果A=B，R就被称作A上的一个二元关系
  - 二元关系实际上是表述了集合A和集合B中元素的某种关联性
    - 例如足球联赛中每轮的对阵情况就是足球俱乐部集合上的一个二元关系
  - 如果$< a, b > \in R$，
  - 则 ‘a是b关于R的前件’ ，而 ‘b是a关于R的后件’

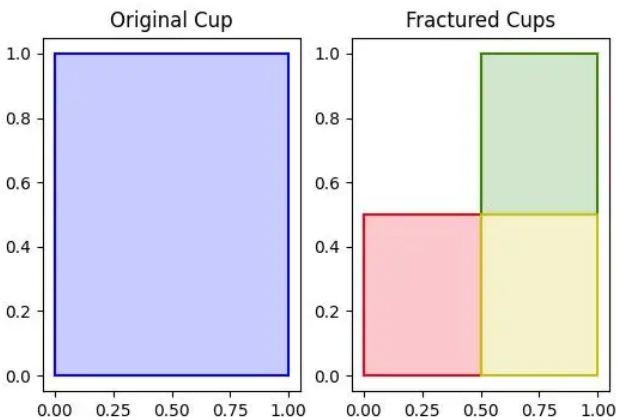
多媒体语义分析

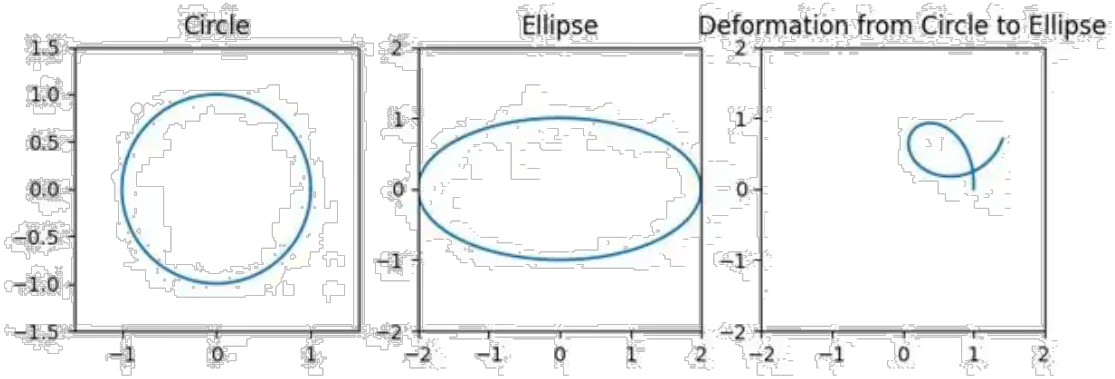
二元关系及性质

- **二元关系基本性质**
  - 设R是集合A上的二元关系：
  - 自反性：对于每个$a\in A$，都有$\langle a,a\rangle\in R$
  - 反自反性：对于所有的$a\in A$，都有$\langle a,a\rangle\notin R$
  - 对称性：如果有$\langle a,b\rangle\in R$，则必有$\langle b,a\rangle\in R$
  - 反对称性：
    - $\langle a,b\rangle\in R$且$\langle b,a\rangle\in R$，则必有$a=b$
    - $\langle a,b\rangle\in R$且$\langle b,a\rangle\notin R$
  - 传递性：如果有$\langle a,b\rangle\in R$且$\langle b,c\rangle\in R$，则必有$\langle a,c\rangle\in R$

二元关系及性质

- **常见的二元关系**
  - 实数集合上的 =, ≤，≥ 的关系；
  - 集合的 =， ⊆， ⊂ 的关系；
  - 三角形的全等关系，相似关系等；
  - 拓扑空间的同胚关系(庞加莱猜想)；
  - 校友关系，用户名和密码
  - ……





二元关系及性质

- **等价关系 (equivalence relation)**
  - 同时满足自反性，对称性和传递性的关系
  - 如果R是等价关系，且$<a,b> \in R$，则称a等价于b，记作 $a \sim b$
  - 设R是非空集合A上的等价关系，则A中相互等价的元素所构成的不相交的子集成为A的R等价类
  - 上面提到的：实数集合上的相等关系，集合的相等关系，校友关系，三角形全等或相似关系，以及拓扑空间的同胚关系，均是等价关系

二元关系及性质

- **偏序关系 (partial order relation)**
  - 同时满足自反性（允许自己和自己的关系），反对称性和传递性的关系
  - 如果R是集合A上的偏序关系，则称<A,R>为偏序集合，也可记作<A,≤ >
  - 偏序集合<A,≤ >中的 x,y ∈A，如果有x≤y或y≤x成立，则称x,y是可比较的；
  - 实数集上的 ≤ 关系，集合的 ⊆关系，均是偏序关系

二元关系及性质

- **拟序关系 (quasi order relation)**
  - 同时满足反自反性，反对称性和传递性的关系
  - 也被称作严格偏序关系(strict partial order)
  - 实数集上的 $<$ 关系，集合的 $\subset$ 关系，均是严格偏序关系.

二元关系及性质

- **全序关系 (total order relation)**
  - 如果R是集合A上的偏序关系，且A中的任意两个元素x,y都可比较，则称R是A上的全序关系；
  - 定义了全序关系的集合称为全序集；
  - 实数集合上的 $\leq$ 关系是全序关系.

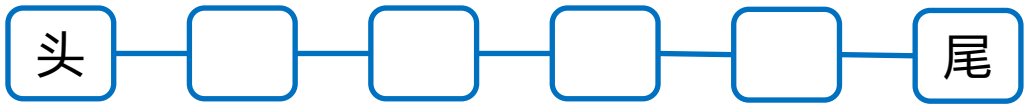
思考题：举例，不是全序关系的偏序关系？

本节内容

- C++简单回顾
- 二元关系及性质
- 数据的逻辑与存储结构
- 数据类型与抽象数据类型

数据的逻辑及存储结构

- 数据逻辑结构：
  - 数据(D)以及数据元素之间的关系(R)，记作 B=(D,R)；
- 线性结构：头元素仅有一个后继元素，尾元素仅有一个前驱元素，其他元素有且仅有一个前驱元素、一个后继元素



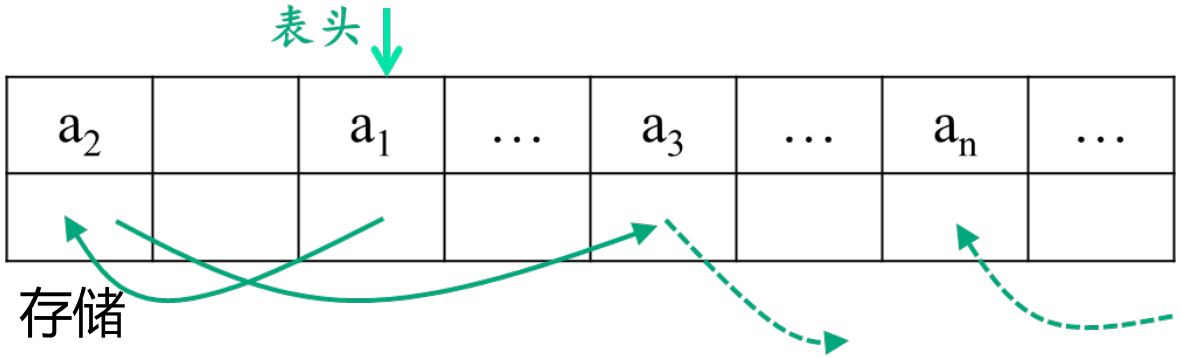
- 非线性结构

数据的逻辑及存储结构

- 数据存储结构(物理结构)：数据元素及其关系在计算机中的表示(representation).
- 顺序存储：数据元素按逻辑顺序依次存放于连续存储单元中:

a_1	a_2	…	a_{n-1}	a_n
0	1	…	n-2	n-1

- 链式存储：通过数据元素存储地址指针表达元素之间的逻辑关系



- 索引与散列（Hashing）存储

本节内容

- C++简单回顾
- 二元关系及性质
- 数据的逻辑与存储结构
- 数据类型与抽象数据类型

抽象数据类型

数据类型	关键字	内存占用	取值范围
整数	byte	1	-128~127
	short	2	-32768~32767
	int(默认)	4	-2的31次方到2的31次方-1
	long	8	-2的63次方到2的63次方-1
浮点数	float	4	负数：-3.402823E+38到-1.401298E-45 正数： 1.401298E-45到3.402823E+38
	double(默认)	8	负数：-1.797693E+308到-4.9000000E-324 正数：4.9000000E-324 到1.797693E+308
字符	char	2	0-65535
布尔	boolean	1	true，false

- **C++中的数据类型**
  - 五种基本数据类型：
    - 无值型(void)，字符型(char)，整型(int)
    - 单精度浮点型(float)，双精度浮点型(double)
  - 可在基本数据类型前添加修饰符：signed, long等
  - 各种类型均有定义相关操作，如 + - * / % 等
- **数据类型**
  - 性质相同的元素的集合以及定义在其上的一组操作
  - ‘封装好’ 的数据结构，用户无需知道计算机内的表示和实现细节
  - 高级语言中的数据类型分为原子类型和结构类型

抽象数据类型

- **ADT: Abstract Data Type**
- **ADT是抽象层次更高，范畴更广的数据类型**
  - 定义：描述数据的逻辑结构和所允许的操作
  - 实现：完成功能，其方式取决于平台、语言等
- **ADT的优点：**
  - 清晰易读，便于开发、调试、扩展与维护
  - 增加了软件的可复用度
  - 通过数据封装，分离使用和实现，增强关键数据的安全性

抽象数据类型

- **ADT例子：三角形**

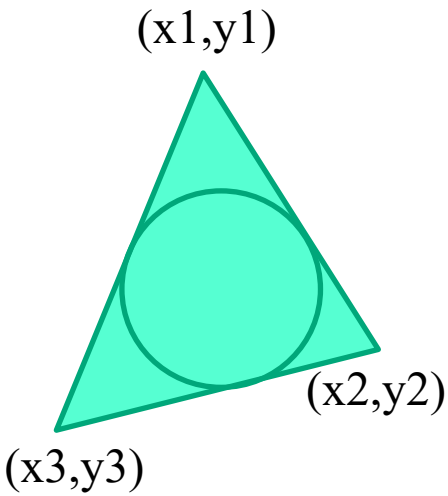
```
ADT Triangle {
    数据对象: D={x1,x2,x3,y1,y2,y3| ∈R}
    数据关系: R={<x1,y1>,<x2,y2>,<x3,y3>|三点坐标}
    基本操作:

    CreateTriangle(x1,x2,x3,y1,y2,y3)
            操作结果: 给出三点坐标, 构造三角形

    Area(Triangle t, &a)
            初始条件: 三角形存在
            操作结果: 计算三角形面积, 以a返回;

    Perimeter(Triangle t, &p)
            初始条件: 三角形存在
            操作结果: 计算三角形周长, 以p返回;

    InscribedCircle(Triangle t, &Cx, &Cy, &r)
            初始条件: 三角形存在
            操作结果: 计算三角形内切圆, 返回圆心坐标及半径;
}ADT Triangle
```



抽象数据类型

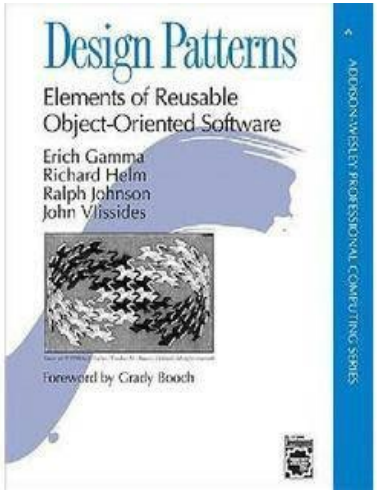
- **ADT的描述规范**

```
ADT 抽象数据类型名 {
    数据对象：<数据对象定义>
    数据关系：<数据关系定义>
    基本操作：
    基本操作名（参数列表）
        初始条件：<初始条件描述>
        操作结果：<操作结果描述>
    … …
}ADT 抽象数据类型名
```

- **基本操作参数列表**
  - 赋值参数提供输入值
  - 引用参数用&修饰，用于结果返回

抽象数据类型

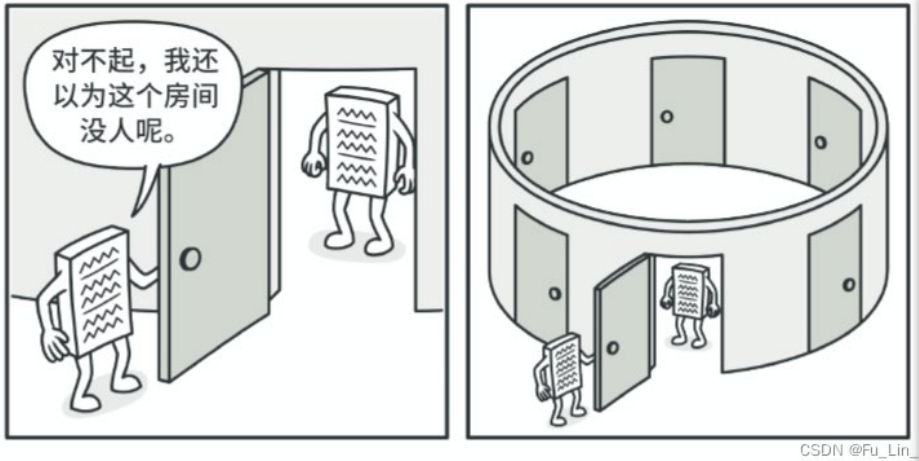
- **编程范式 (paradigm)**
  - 面向过程编程(Pascal, C)
  - 基于对象编程(OO, but encapsulation only)
  - 面向对象编程(C++, Java)
  - 面向切面编程(AspectJ)
  - 函数式编程(F#,LISP)
  - ……
- **ADT与面向对象**
  - 面向对象语言提供了良好的封装特性
  - 继承(inheritance)与多态(polymorphism)方便模块的复用
  - ADT出发可以很容易实现面向对象编程
- **设计模式(Design Pattern)**
  - 例子：Singleton-只能创建一个对象的类



GpF：Gang of Four

设计模式通过最大程度的利用面向对象的特性，诸如利用继承、多态，对责任进行分离、对依赖进行倒置，面向抽象，面向接口，最终设计出灵活、可扩展、可重用的类库、组件，乃至于整个系统的架构

Singleton 单例模式



- 原因：控制某些共享资源（例如数据库或文
- 运作方式：如果你创建了一个对象，过一会
  之前已创建的对象，而不是一个新对象
- 实现：
  - 将默认构造函数设为私有， 防止其他对象使用
  - 新建一个静态构建方法作为构造函数。该函数
    存在一个静态成员变量中。此后所有对于该函
- 如果需要更加严格地控制全局变量，可以使
- 单例模式与全局变量不同，它保证类只存在
  方式替换缓存的实例

```
#include <iostream>
using namespace std;

class A
{
private:
    A(): data(10) { cout << "A" << endl; }
    ~A(){ cout << "~A" << endl; }
public:
    static A& Instance()
    {
        static A a;
        return a;
    }
    void Print()
    {
        cout << data << endl;
    }
private:
    int data;
};

int main(int argc, char** argv)
{
    A &ra = A::Instance();
    ra.Print();
}
```

谢　谢！

北京科技大学
计算机与通信工程学院
杜　冰